



《实用Python编程》

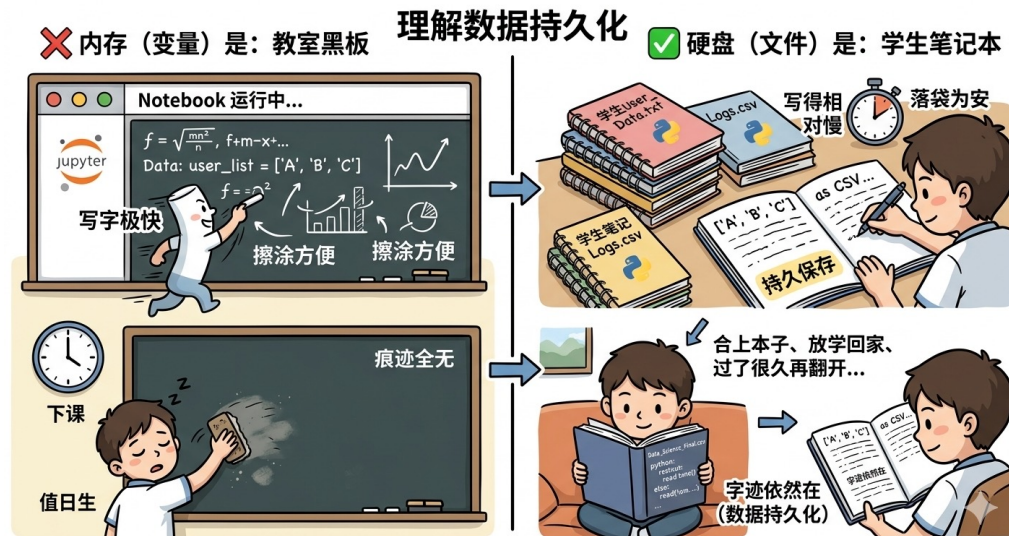
第4讲：数据存储与网上数据抓取-A

实用Python编程 教研组

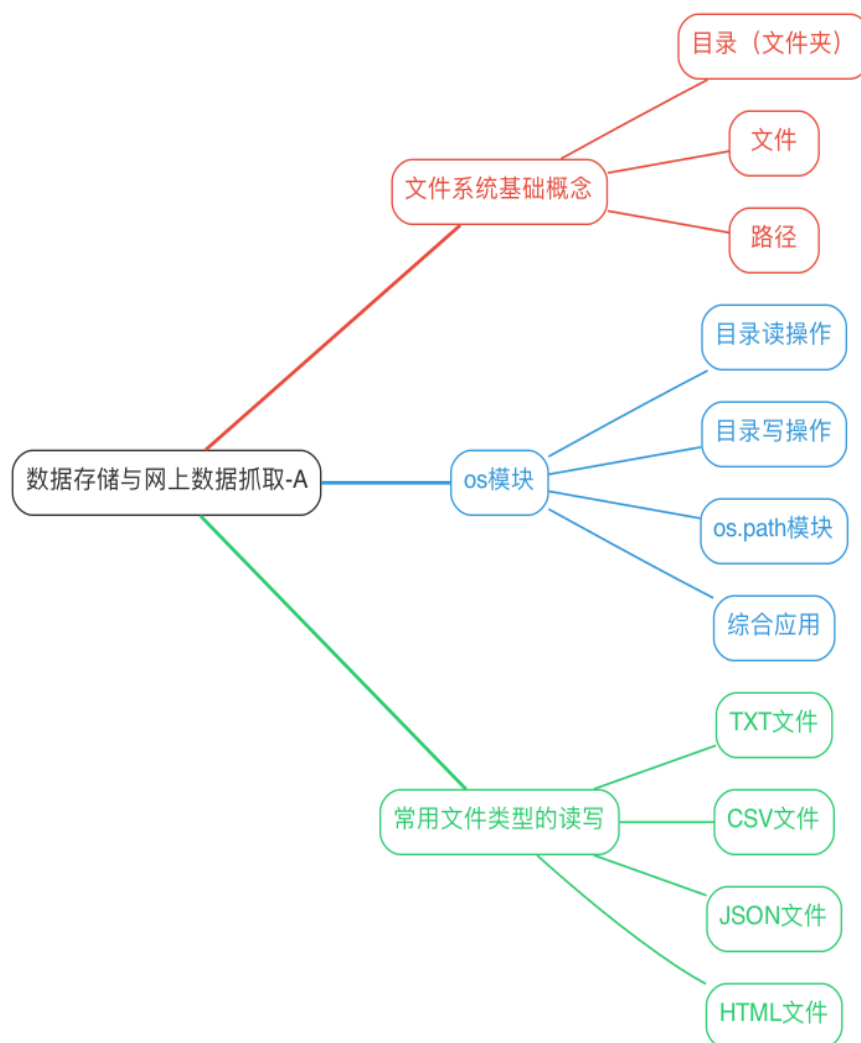


数据存得下来，才能被复用

- 变量中的数据是临时的，会随着Notebook关闭而消失
- 数据的长久保存与复用依赖文件读写



提纲



文件系统基础概念

- 数据储存位置与时效性

位置	存储对象	时效性
内存	程序执行过程中产生的数据（变量）	临时
磁盘	文件	长久

临时的

```
1 import json
2 name = "Jims Hacker"
3 age = 52
4 data = {'name': name, 'age': age}
5 data
```

```
{'name': 'Jims Hacker', 'age': 52}
```

```
1 with open('data.json', 'w') as fw:
2     json.dump(data, fw)
```

↓ 存入文件（长久的）

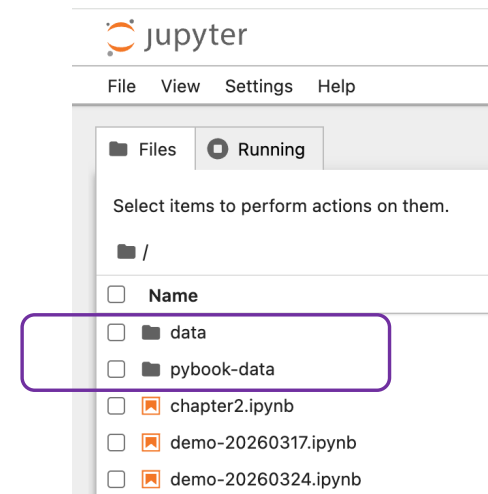
```
{"name": "Jims Hacker", "age": 52}
```

文件系统基础概念

- **目录**(Directory), 也称为**文件夹**(Folder)
 - 计算机用来组织文件的逻辑结构
 - 目录可以为空, 也可以包含子目录和文件
 - 但文件中不能再包含文件或目录

一个文件夹就好比一个房间, 目录就是进入该房间的门

- 文件: 房间里的物件
- 子文件夹: 房间里的房间, 子目录就是通过该房间的门





文件系统基础概念

- 目录的层级结构

给定一个目录，包含它的称为它的上级目录，被它包含的为其子目录
没有上级目录的，称为根目录

当前目录	上级目录	子目录	根目录
C:\Users\course	C:\Users	C:\Users\course\pycode	C:\



文件系统基础概念

- **路径 (Path)**
 - 描述了文件或目录所在位置
 - 从根目录开始的完整路径称为**绝对路径** (Absolute Path)
 - 从当前位置开始的路径称为**相对路径** (Relative Path)
 - 路径分隔符
 - Linux / Mac系统使用**斜杠** ‘/’
 - Windows系统使用**反斜杠** ‘\’
- 对于 C:\Users\course\pycode 路径下的 hello.ipynb 文件
 - 绝对路径是 C:\Users\course\pycode\hello.ipynb
 - 若当前位置是 pycode, 则相对路径是 hello.ipynb
 - 若当前位置是 course, 则相对路径是 pycode\hello.ipynb

如何获得当前位置? (示例代码1.1)



os模块：目录操作

- 使用前先通过语句 `import os` 导入
 - os为Python内置模块，无需额外安装
- 常用读操作
 - 获取当前工作目录 `os.getcwd()`
 - 改变当前工作目录 `os.chdir()`
 - 罗列指定目录中的内容 `os.listdir()`

os的读操作既不会创建新目录，也不会修改或删除现有目录



os模块：常用读操作

- `os.listdir`会列出指定目录包含的内容（文件、文件夹/子目录）
 - 但不会再列出子目录中的内容

```
os.listdir('.') # 列出当前目录下的所有文件和文件夹
```

- `os.walk` 可以遍历到指定目录中的所有内容（含子目录中的）



os模块：常用读操作

- `os.walk`在循环遍历过程中每次返回一个“三元组”
 - `dirpath`: 当前正在遍历的那个文件夹的**绝对路径** (字符串)
 - `dirnames`: 当前文件夹下包含的**所有子文件夹名字** (列表)
 - `filenames`: 当前文件夹下包含的**所有文件名字** (列表)

```
for dirpath, dirnames, filenames in os.walk(root_path):  
    print(f"当前正在扫描的目录: {dirpath}")  
    print(f"发现的子文件夹: {dirnames}")  
    print(f"发现的文件: {filenames}")  
    print("-" * 50)
```



os模块：常用写操作

- 创建单级目录 `os.mkdir`
- 创建多级目录 `os.makedirs`

`os.mkdir('dir1')` # 在当前目录下新建一个名为 `dir1` 的文件夹，若同名文件夹存在，则报错

`os.makedirs('dir1', exist_ok=True)` # 同名文件夹存在时不报错

`os.makedirs('dir1/dir2/dir3')` # 在相对路径为`dir1/dir2`的目录中新建一个名为`dir3`的文件夹。
若`dir1`或`dir2`不存在，则会在对应层级自动创建



os模块：常用写操作

- 删除单级空目录 `os.rmdir`
 - 删除多级空目录 `os.removedirs`
 - 删除文件 `os.remove`
 - 重命名文件或文件夹 `os.rename(old_name, new_name)`
- } 若目录不空，报错
`OSError: [Errno 66]`
`Directory not empty`



课堂练习3-1-1 创建多级目录再删除

先创建

```
os.makedirs('test1/test2/test3', exist_ok=True)  
open('test1/test2/test3/a.txt', 'w').write('12345')
```

再删除 'test2'

最后将 'test1' 再重命名为 'test1-renamed'



os.path模块：路径操作

- 使用前先通过 `import os` 或者 `import os.path` 导入
- 存在性判断
 - 判断路径是否存在 `os.path.exists`
 - 判断路径所指的是否为文件 `os.path.isfile`
 - 判断路径所指的是否为目录（文件夹） `os.path.isdir`
- 获取路径
 - 获取绝对路径 `os.path.abspath`

尝试获取一个不存在的文件的绝对路径：`os.path.abspath('xyz.123')`
观察一下代码的执行结果，是报错，还是返回一个不存在的路径？



os.path模块：路径操作

- 路径拼接 `os.path.join`
- 路径拆分
 - 拆分为上级目录与文件名 `os.path.split`
 - 拆分为文件名与扩展名 `os.path.splitext`

这些函数会根据当前操作系统
自动选择合适的路径分隔符





读取文件基础信息

- 文件大小 `os.path.getsize`
- 最后修改时间 `os.path.getmtime` # m是modify
- 最后访问时间 `os.path.getatime` # a是access



综合应用：遍历目录统计文件信息

- 需求：在一个指定的文件夹下找出所有Word文档并按照最后修改时间从后到前排序输出

任务拆解	实施方案
获取指定文件夹下的所有文件	使用 <code>os.walk</code> 函数访问多级嵌套的子文件夹
判断是否为Word文件	用字符串的 <code>endswith</code> 函数检查是否以 <code>".docx"</code> 结尾
获取文件最后修改时间	调用 <code>os.path.getmtime</code> 函数返回修改时间戳
所有Word文件按修改时间从后到前排序	用列表汇集每个Word文件及其修改时间戳，并调用列表内置的 <code>sort</code> 方法



综合应用：遍历目录统计文件信息

- 列表排序时
 - 如果其元素是多元组，默认会选取多元组的第一项作为排序值
 - 将修改时间戳 `mtime` 作为二元组的第一项，便于排序操作
 - 也可以用`lambda`操作（2.4.3节）
- 默认情况下`sort`方法返回的是升序（即按照修改时间从旧到新）而任务要求是降序，因此需要将`sort`方法的`reverse`参数设为`True`



综合应用：遍历目录统计文件信息

- 完整实现

```
1 import os, time
2
3 target_dir = '/Users/xirong/workspace' # 指定待查找的目录/文件夹
4 res = [] # 创建一个空列表, 用于收集 (修改时间戳, Word文件路径) 二元组
5
6 for root, dirs, files in os.walk(target_dir):
7     for fname in files:
8         if fname.endswith('.docx') is False: # 跳过非Word文件
9             continue
10        full_path = os.path.join(root, fname) # 获得当前文件的完整路径
11        mtime = os.path.getmtime(full_path)
12        res.append((mtime, full_path)) # 将修改时间放在前面, 以便排序
13
14 # 遍历结束, 进行排序
15 res.sort(reverse=True) # 激活reverse参数, 让列表降序排序
16
17 # 输出结果
18 for (mtime, full_path) in res:
19     readable_time = time.ctime(mtime) # 将时间戳转为简单易读格式
20     print(full_path, readable_time)
```

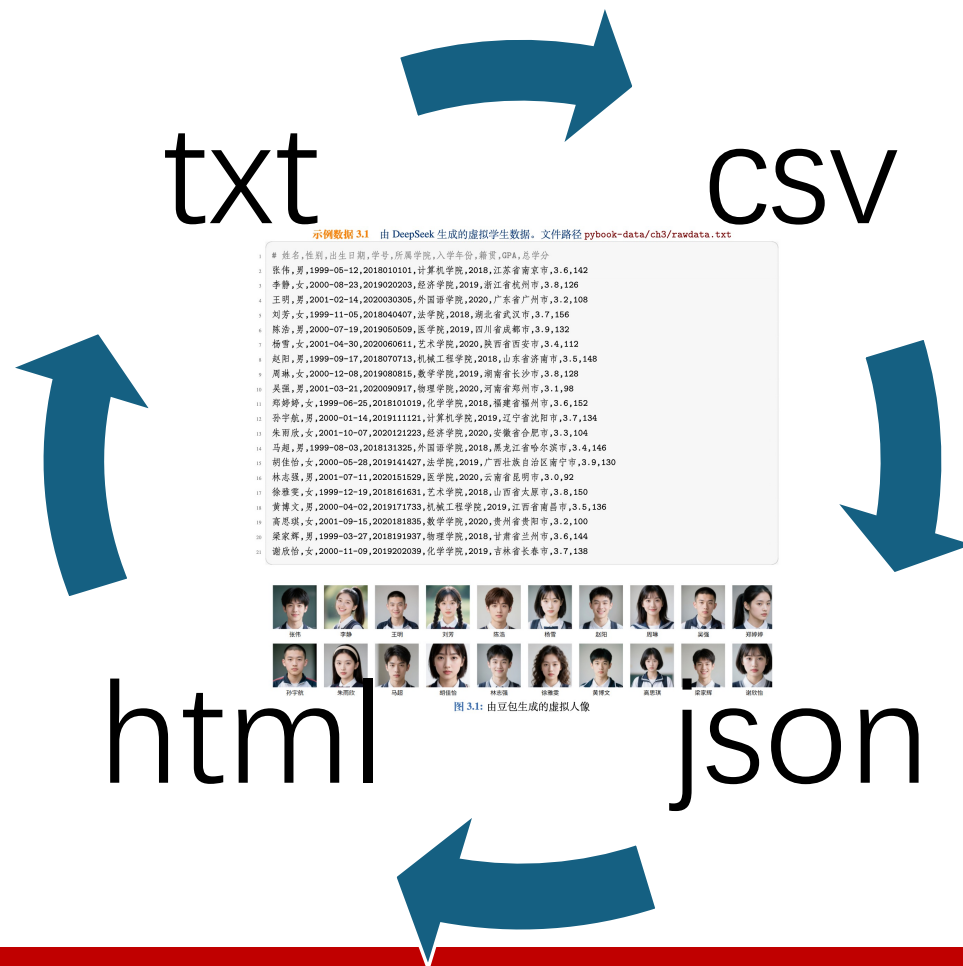


课堂练习3-1-2 文件统计

在你的Jupyter Notebook当前工作路径下将ch3-txt.zip解压缩

- 1) 先通过`os.path.exists`检查数据是否就绪
- 2) 统计ch3-txt目录下包含的文件夹数量和txt文本文件数量
- 3) 进一步，统计文件里存放了两位数的文件数

常用文件类型的读写操作



文本文件

- 纯文本 txt
 - 结构化 csv, json, html
- ## 二进制文件（进阶内容）
- jpg, excel, word, sqlite



TXT纯文本文件

- 纯文本文件（以.txt为扩展名）是最基础的文件类型
 - 以字符序列形式存储数据，可读性强
 - 适合存储结构简单的文本数据

```
jupyter rawdata.txt Last Checkpoint: 1 minute ago
File Edit View Settings Help

1 # 姓名,性别,出生日期,学号,所属学院,入学年份,籍贯,GPA,总学分
2 张伟,男,1999-05-12,2018010101,计算机学院,2018,江苏省南京市,3.6,142
3 李静,女,2000-08-23,2019020203,经济学院,2019,浙江省杭州市,3.8,126
4 王明,男,2001-02-14,2020030305,外国语学院,2020,广东省广州市,3.2,108
5 刘芳,女,1999-11-05,2018040407,法学院,2018,湖北省武汉市,3.7,156
6 陈浩,男,2000-07-19,2019050509,医学院,2019,四川省成都市,3.9,132
7 杨雪,女,2001-04-30,2020060611,艺术学院,2020,陕西省西安市,3.4,112
8 赵阳,男,1999-09-17,2018070713,机械工程学院,2018,山东省济南市,3.5,148
9 周琳,女,2000-12-08,2019080815,数学学院,2019,湖南省长沙市,3.8,128
10 吴强,男,2001-03-21,2020090917,物理学院,2020,河南省郑州市,3.1,98
```

pybook-data/ch3/rawdata.txt

<https://github.com/li-xirong/python-book/blob/main/pybook-data/ch3/rawdata.txt>

TXT纯文本文件

- 从TXT文件中读取数据

- `fr=open(input_file)` 以读模式打开一个文件，并返回一个文件对象`fr`
- `fr.readlines()` 按行一次性读取整个文件，返回一个字符串列表
- `fr.read()` 一次性读取整个文件，返回一个字符串
- `fr.close()` 关闭文件

```
1 input_file = "pybook-data/ch3/rawdata.txt"
2 output_file = "students.txt"
3 # 读文件
4 fr = open(input_file) # 以读模式 (r) 获得一个文件对象
5 lines = fr.readlines() # 逐行读取数据
6 fr.close() # 关闭该文件对象
```



TXT纯文本文件

- 数据解析

- `line.strip()` # 去掉换行符(`\r`, `\n`)以及行尾的空格
- `line.split(',')` # 以逗号为分隔符, 分割出各个字段
- `','.join(elms)` # 用逗号拼接各个字段

```
7 # 数据解析
8 lines = [x for x in lines if not x.startswith('#')] # 去掉以#开头的注释行
9 records = []
10 for line in lines:
11     line = line.strip() # 去掉换行符
12     elms = line.split(',') # 以逗号为分隔符, 获得各个字段
13     newline = ','.join(elms) # 将各个字段再以逗号为分隔符拼接回一个字符串
14     records.append(newline)
```

TXT纯文本文件

• 将数据写入TXT文件

- `fw = open(output_file, "w")` # 以写模式打开文件，并返回一个文件对象fw
- `fw.write()` # 写入文件
- `fw.close()` # 关闭文件

```
1 input_file = "pybook-data/ch3/rawdata.txt"
2 output_file = "students.txt"
3 # 读文件
4 fr = open(input_file) # 以读模式 (r) 获得一个文件对象
5 lines = fr.readlines() # 逐行读取数据
6 fr.close() # 关闭该文件对象
7 # 数据解析
8 lines = [x for x in lines if not x.startswith('#')] # 去掉以#开头的注释行
9 records = []
10 for line in lines:
11     line = line.strip() # 去掉换行符
12     elems = line.split(',') # 以逗号为分隔符，获得各个字段
13     newline = ','.join(elems) # 将各个字段再以逗号为分隔符拼接回一个字符串
14     records.append(newline)
15 # 写文件
16 fw = open(output_file, "w") # 以写模式 (w) 打开一个新的文件对象
17 for line in records:
18     fw.write(line + '\n') # 加上换行符，否则所有记录会写到一行
19 fw.close()
```



结构化文本文件

- CSV、JSON和HTML文件存储文本数据
 - 可视为带特殊扩展名的纯文本文件
- 具备结构化的特点，可用相应Python库将文本数据转为结构化数据

CSV文件

- CSV(Comma-Separated Values)是一种轻量级结构化文本格式,大量用于表格数据的存储
 - 特点是以纯文本形式表示二维表格,每行对应一条记录,每条记录的不同字段使用特定分隔符(通常是英文逗号)隔开
 - csv文件的第一行通常是用于解释各列含义的表头
 - 不含标注表头也符合CSV规范

jupyter rawdata.csv Last Checkpoint: 1 minute ago

File Edit View Settings Help

Delimiter: ,

	姓名	性别	出生日期	学号	所属学院	入学年份	籍贯	GPA
1	张伟	男	1999-05-12	2018010101	计算机学院	2018	江苏省南京市	3.6
2	李静	女	2000-08-23	2019020203	经济学院	2019	浙江省杭州市	3.8
3	王明	男	2001-02-14	2020030305	外国语学院	2020	广东省广州市	3.2
4	刘芳	女	1999-11-05	2018040407	法学院	2018	湖北省武汉市	3.7
5	陈浩	男	2000-07-19	2019050509	医学院	2019	四川省成都市	3.9
6	杨雪	女	2001-04-30	2020060611	艺术学院	2020	陕西省西安市	3.4
7	赵阳	男	1999-09-17	2018070713	机械工程学院	2018	山东省济南市	3.5
8	周琳	女	2000-12-08	2019080815	数学学院	2019	湖南省长沙市	3.8
9	吴强	男	2001-03-21	2020090917	物理学院	2020	河南省郑州市	3.1
10	郑婷婷	女	1999-06-25	2018101019	化学学院	2018	福建省福州市	3.6

TXT转CSV

- 使用前先通过 `import csv` 导入
- 使用 `csv.writer` 创建一个csv写对象

```
1 import csv
2 # 定义表头
3 headers = ["姓名", "性别", "出生日期", "学号", "所属学院", "入学年份", "籍贯", "GPA", "总学分"]
4
5 # 从students.txt中读取数据
6 with open('students.txt', 'r') as fr:          with open(...) as fr 省去手动关闭文件对象的麻烦
7     lines = fr.readlines() # 读取所有行
8     records = [x.strip() for x in lines if x.strip()] # 去掉换行符、空行
9
10 # 将数据写入CSV文件
11 with open('students.csv', 'w', newline='') as fw:
12     writer = csv.writer(fw)
13     # 写入表头
14     writer.writerow(headers)
15     # 写入数据行
16     for student in records:
17         writer.writerow(student.split(','))
```

csv会自动换行，所以调用open函数时，换行符参数newline要设置成空字符串



JSON文件

- JSON(JavaScript Object Notation)是一种简洁轻量、易于解析的结构化文本格式，广泛用于列表、键-值对等数据的存储

最常用的对象数组 (array of objects) 格式

```
[  
{"name": "iPhone 17", "price": 7000},  
{"name": "MacBook Pro", "price": 12000}  
]
```

整个文件以一个方括号 [] 包裹，
里面是多个结构相同的 JSON 对象，
对象之间以逗号隔开



CSV转JSON

- 使用`json.dump`将每个字典对象写入`json`文件
 - 为了正确显示汉字，设置参数`ensure_ascii=False`

```
1 import csv
2 import json
3
4 # 从students.csv中读取数据
5 with open('students.csv', 'r') as f:
6     csv_data = list(csv.reader(f)) # 读取csv数据
7     headers = csv_data.pop(0) # 从数据中将表头列表弹出
8
9 # 将每条学生记录转换为一个字典对象
10 students = [dict(zip(headers, row)) for row in csv_data]
11
12 # 将数据写入JSON文件
13 with open('students.json', 'w') as fw:
14     json.dump(students, fw, ensure_ascii=False)
```

```
[{"姓名": "张伟", "性别": "男", "出生日期": "1999-05-12",
"学号": "2018010101", "所属学院": "计算机学院", "入学年份":
"2018", "籍贯": "江苏省南京市", "GPA": "3.6", "总学分":
"142"}, {"姓名": "李静", "性别": "女", "出生日期": "2000-
08-23", "学号": "2019020203", "所属学院": "经济学院", "入
学年份": "2019", "籍贯": "浙江省杭州市", "GPA": "3.8", "总
学分": "126"}, {"姓名": "王明", "性别": "男", "出生日期":
"2001-02-14", "学号": "2020030305", "所属学院": "外国语学院", "入学年份": "2020", "籍贯": "广东省广州市", "GPA":
"3.2", "总学分": "108"}, ...]
```

HTML文件

- 网页是用html语言写的
 - 超文本标记语言 (HyperText Markup Language, html)

```
<html>
<body>
<h1>hello world</h1>
<h3>hello world</h3>
</body>
</html>
```

存为 hello.html
并用浏览器看打开看看



hello world

hello world



HTML文件

- html标签以尖括号括起来，开始标签与结束标签成对出现
 - 开始标签 `<html>` `<body>` `<p>` `<font>`
 - 结束标签 `</html>` `</body>` `</p>` `</font>`
 - 嵌套：一对标签可以被包含在另一对标签中
 - 特殊标签，只能出现一次：html, head, title, body

```
<html>
<body>
hi, <span style="font-style: italic; font-size: 58px; font-weight:
bold;">this</span> is <span style="color: blue; font-size: 28px">my</span>
first html <span style="color: red; font-size: 20px"> page </span>
</body>
</html>
```

hi, *this* is my first html page

JSON转HTML

- HTML本质上就是文本
 - 以f-string构造HTML字符串，再以文本文件的方式保存

```
1 import json
2 with open('students.json') as fr:
3     data = json.load(fr)
4     # 构造表头，列表为相应记录的相应字段
5     table = ''
6     column_names = ['照片'] + list(data[0].keys())
7     for col in column_names:
8         table += f'<th>{col}</th>'
9     table = f'<tr>{table}</tr>' # 表格头是表格的第1行，以<tr>开头，</tr>结尾
10    # 一条学生记录对应表格的一行
11    table_content = []
12    for record in data:
13        img_path = f'pybook-data/ch3/portraits/{record["学号"]}.png'
14        new_row = f'<td></img></td>'
15        new_row += ''.join([f'<td>{record[col]}</td>' for col in column_names[1:]])
16        table_content.append(f'<tr>{new_row}</tr>')
17    table = '<table>' + table + '\n' + '\n'.join(table_content) + '</table>'
18    style = '<style>table, th, td {border: 1px solid black; border-collapse: collapse;}</style>'
19    html_content = f'<html>\n<head>{style}</head>\n<body>{table}</body>\n</html>'
20    # 保存为 .html 文件
21    with open('students.html', 'w') as fw:
22        fw.write(html_content)
```

<table>标签定义HTML的表格

- <th>标签定义表头，即学生记录的相应字段
- <tr>标签定义表格的一行
- <td>标签定义表格的一个单元格

标签定义图片

- src: 图片路径
- width: 图片的显示宽度
- height: 图片的显示高度

置于<head>内的<style>标签定义html各元素的显示样式

JSON转HTML

- 在浏览器中打开students.html

```

1 import json
2 with open('students.json') as fr:
3     data = json.load(fr)
4     # 构造表头, 列表为相应记录的相应字段
5     table = ''
6     column_names = ['照片'] + list(data[0].keys())
7     for col in column_names:
8         table += f'<th>{col}</th>'
9     table = f'<tr>{table}</tr>' # 表头是表格的第1行, 以<tr>开头, </tr>结尾
10    # 一条学生记录对应表格的一行
11    table_content = []
12    for record in data:
13        img_path = f'pybook-data/ch3/portraits/{record["学号"]}.png'
14        new_row = f'<td></img></td>'
15        new_row += ''.join([f'<td>{record[col]}</td>' for col in column_names[1:]])
16        table_content.append(f'<tr>{new_row}</tr>')
17    table = f'<table>' + table + '\n' + '\n'.join(table_content) + f'</table>'
18    style = f'<style>table, th, td {border: 1px solid black; border-collapse: collapse;}</style>'
19    html_content = f'<html>\n<head>{style}</head>\n<body>{table}</body>\n</html>'
20    # 保存为 .html 文件
21    with open('students.html', 'w') as fw:
22        fw.write(html_content)

```

试试将第18行改为style="", 即使用默认样式

照片	姓名	性别	出生日期	学号	所属学院	入学年份	籍贯	GPA	总学分
	张伟	男	1999-05-12	2018010101	计算机学院	2018	江苏省南京市	3.6	142
	李静	女	2000-08-23	2019020203	经济学院	2019	浙江省杭州市	3.8	126
	王明	男	2001-02-14	2020030305	外国语学院	2020	广东省广州市	3.2	108
	刘芳	女	1999-11-05	2018040407	法学院	2018	湖北省武汉市	3.7	156
	陈浩	男	2000-07-19	2019050509	医学院	2019	四川省成都市	3.9	132
	杨雪	女	2001-04-30	2020060611	艺术学院	2020	陕西省西安市	3.4	112
	赵阳	男	1999-09-17	2018070713	机械工程学院	2018	山东省济南市	3.5	148

HTML文件解析

- 第三方库BeautifulSoup可从HTML中提取目标信息
 - 需要借助**解析器**(Parser)解析HTML文件
 - 通常使用python自带的**html.parser**
 - 两种初始化BeautifulSoup对象的方法
 - BeautifulSoup(文件对象, 'html.parser')
 - BeautifulSoup(字符串, 'html.parser')

```
1 from bs4 import BeautifulSoup
2
3 html_file = "students.html"
4
5 # 方法1: 基于html文件对象的初始化
6 with open(html_file) as fr:
7     soup_1 = BeautifulSoup(fr, 'html.parser')
8
9 # 方法2: 基于html文件内容的初始化
10 with open(html_file) as fr:
11     soup_2 = BeautifulSoup(fr.read(), 'html.parser')
12
13 print(soup_1 == soup_2) # 判断内容是否一致。输出True
```

HTML文件解析

- BeautifulSoup解析和操作HTML文档的主要接口是一个名为`bs4.element.Tag`的类，用于表示HTML文档中的标签元素
 - `Tag`对象可通过BeautifulSoup对象的两个标签查找方法`find`和`find_all`获得

表 3.3: BeautifulSoup 的标签查找方法

方法	输入参数	返回值	适用场景	调用示例
<code>find</code>	标签名	首个匹配到的标签	获取单个元素	<code>soup.find('table')</code> # 从 html 文件中提取表格
<code>find_all</code>	标签名	匹配到的标签列表	获取所有同类元素	<code>soup.find('table').find_all('tr')</code> # 提取表格中的所有行

HTML文件解析

Tag类的常用属性

- name 标签名
- text 标签及其所有子标签的文本内容
- contents 标签的直接子节点列表
- attrs 标签自身属性的字典

示例代码 3.9 读取bs4.element.Tag类的常用属性

```
1 soup = BeautifulSoup('<p>hello <b>world</b> and <em>good day</em></p>', 'html.parser')
2 tag = soup.find('p')
3 print(tag.name, tag.text, tag.contents, tag.attrs) # 输出 p hello world and good day ['hello ', <b>
    world</b>, ' and ', <em>good day</em>] {}
4 soup = BeautifulSoup('', 'html.parser')
5 tag = soup.find('img')
6 print(tag.name, tag.text, tag.contents, tag.attrs) # 输出 img [] {'src': 'demo.jpg'}
```



课堂练习3-1-3 HTML文件解析

给定以下HTML片段，完成相应解析任务：

```
<table>
  <tr><th>姓名</th><th>GPA</th></tr>
  <tr><td>张伟</td><td>3.6</td></tr>
  <tr><td>李静</td><td>3.8</td></tr>
  <tr><td>王明</td><td>3.3</td></tr>
</table>
```

提示：多行文本的字符串使用**三引号**
(Triple Quotes) 作为分界符

- 三个连续的单引号或者三个连续的双引号

- 用 BeautifulSoup 提取所有学生的姓名与GPA，保存到JSON文件
- 找出GPA最高的学生姓名

```
[{"姓名": "张伟", "GPA": 3.6}, {"姓名": "李静", "GPA": 3.8}, {"姓名": "王明", "GPA": 3.3}]
```

课堂练习3-1-4 HTML转TXT

- 通过HTML文件解析，将students.html转存为students.txt
 - 忽略照片

照片	姓名	性别	出生日期	学号	所属学院	入学年份	籍贯	GPA	总学分
	张伟	男	1999-05-12	2018010101	计算机学院	2018	江苏省南京市	3.6	142
	李静	女	2000-08-23	2019020203	经济学院	2019	浙江省杭州市	3.8	126
	王明	男	2001-02-14	2020030305	外国语学院	2020	广东省广州市	3.2	108
	刘芳	女	1999-11-05	2018040407	法学院	2018	湖北省武汉市	3.7	156
	陈浩	男	2000-07-19	2019050509	医学院	2019	四川省成都市	3.9	132
	杨雪	女	2001-04-30	2020060611	艺术学院	2020	陕西省西安市	3.4	112
	赵阳	男	1999-09-17	2018070713	机械工程学院	2018	山东省济南市	3.5	148



张伟,男,1999-05-12,2018010101,计算机学院,2018,江苏省南京市,3.6,142
 李静,女,2000-08-23,2019020203,经济学院,2019,浙江省杭州市,3.8,126
 王明,男,2001-02-14,2020030305,外国语学院,2020,广东省广州市,3.2,108
 刘芳,女,1999-11-05,2018040407,法学院,2018,湖北省武汉市,3.7,156
 陈浩,男,2000-07-19,2019050509,医学院,2019,四川省成都市,3.9,132
 杨雪,女,2001-04-30,2020060611,艺术学院,2020,陕西省西安市,3.4,112
 赵阳,男,1999-09-17,2018070713,机械工程学院,2018,山东省济南市,3.5,148
 周琳,女,2000-12-08,2019080815,数学学院,2019,湖南省长沙市,3.8,128
 吴强,男,2001-03-21,2020090917,物理学院,2020,河南省郑州市,3.1,98
 郑婷婷,女,1999-06-25,2018101019,化学学院,2018,福建省福州市,3.6,152
 孙宇航,男,2000-01-14,2019111121,计算机学院,2019,辽宁省沈阳市,3.7,134
 朱雨欣,女,2001-10-07,2020121223,经济学院,2020,安徽省合肥市,3.3,104
 马超,男,1999-08-03,2018131325,外国语学院,2018,黑龙江省哈尔滨市,3.4,146
 胡佳怡,女,2000-05-28,2019141427,法学院,2019,广西壮族自治区南宁市,3.9,130
 林志强,男,2001-07-11,2020151529,医学院,2020,云南省昆明市,3.0,92
 徐雅雯,女,1999-12-19,2018161631,艺术学院,2018,山西省太原市,3.8,150
 黄博文,男,2000-04-02,2019171733,机械工程学院,2019,江西省南昌市,3.5,136
 高思琪,女,2001-09-15,2020181835,数学学院,2020,贵州省贵阳市,3.2,100
 梁家辉,男,1999-03-27,2018191937,物理学院,2018,甘肃省兰州市,3.6,144
 谢欣怡,女,2000-11-09,2019202039,化学学院,2019,吉林省长春市,3.7,138



进阶内容：二进制文件读写（3.2.3节）

- JPG/PNG图像
- Excel表格
- Word文档
- SQLite数据库